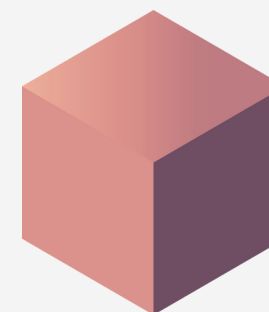
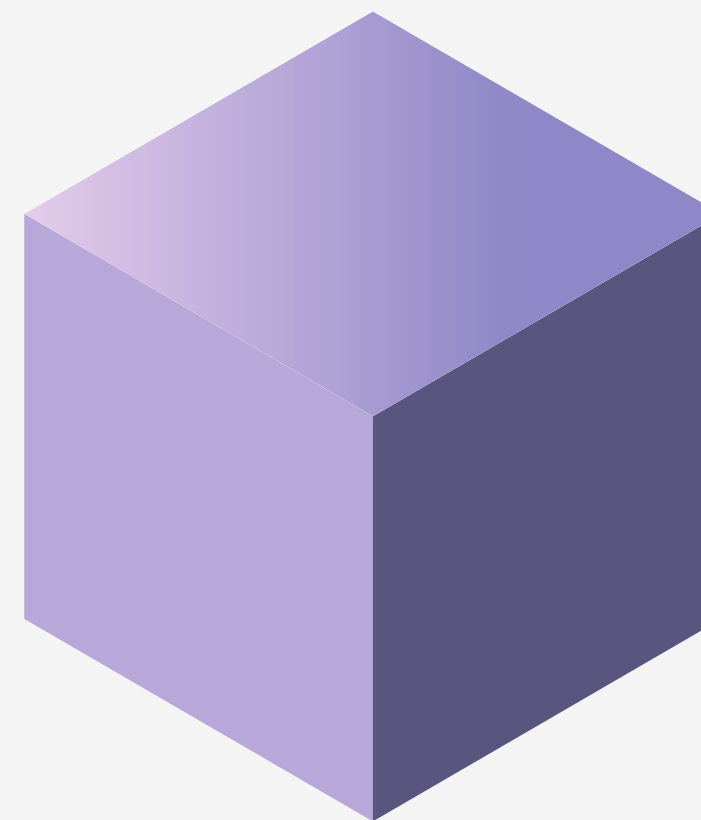




Факултет по Математика и Информатика
ПУ "Паисий Хилендарски"

ДЕКОМПОЗИРАНЕ НА МОНОЛИТНО ПРИЛОЖЕНИЕ (MONOLITH DECOMPOSITION)

Витали Атиас



Кога се налага да
мислим за
декомпозиране на
монолита?

Декомпозиране на монолита

Декомпозиция на монолит (Monolith Decomposition) е процес на модернизация на софтуера, който цели разделяне на големи монолитни приложения на отделни независими микро-услуги (microservices). Тази практика значително подобрява мащабируемостта, гъвкавостта и устойчивостта на модерните приложения.

Основни стъпки в процеса

- Разбиване на монолита на микросървиси - използват се различни декомпозиционни шаблони (decomposition patterns)
- Интеграция на микросървисите - след като всяка услуга е създадена, тя трябва да се интегрира в цялостната микросървисна архитектура. Например, AWS предлага serverless services за интеграция.
- Децентрализиране на базата данни и осигуряване на съхранение на данни за микросървисите - всеки микросървис има собствена база данни.

Видове проекти за модернизация

- **Brownfield проекти** - подразбират разработване или миграция на нови софтуерни системи в контекста на съществуващи приложения или наследен код (legacy systems). Това е често срещаният подход при разбиването на монолити.
- **Greenfield проекти** - подразбират създаването на система "от нулата" без съществуващ код, за напълно нова среда.

Шаблони за декомпозиране на монолити

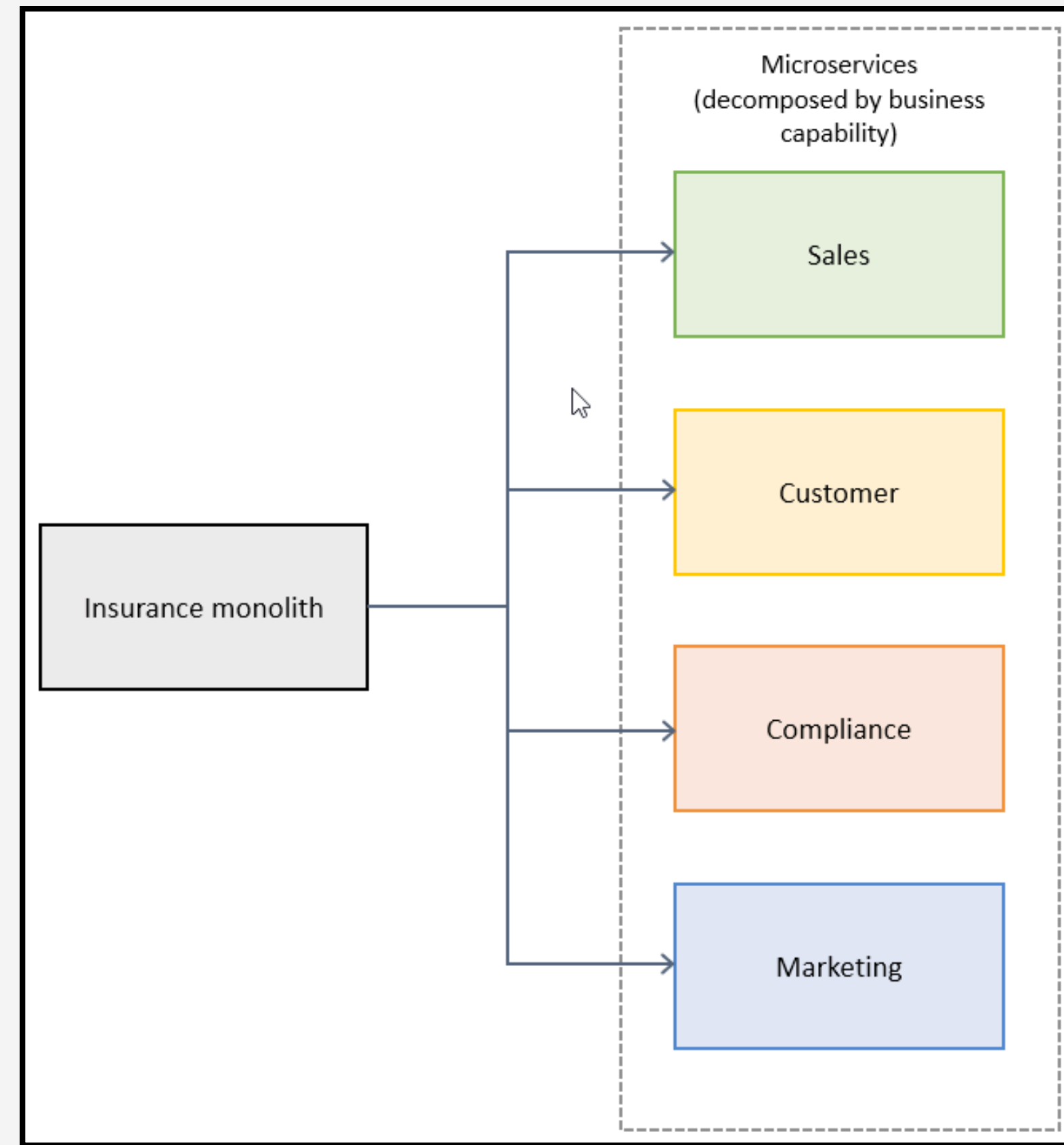
Шаблон	Описание
Според бизнес функционалност (Decompose by business capability)	Разделяне на части според основни бизнес процеси (напр. "Фактуриране", "Плащания").
Според поддомейн (Decompose by subdomain)	Използване на подход Domain-Driven Design (DDD) за изолиране на поддомейни.
Според транзакции (Decompose by transactions)	Разделяне на услуги въз основа на данните и транзакциите между тях.
Един микросървис на екип (Service per team pattern)	Разделяне според отговорностите на екипите, които поддържат конкретни услуги.
Шаблон „Стрейнгълър“ (Strangler fig)	Постепенно заместване на части от монолита с нови микросървиси.
Разклоняване чрез абстракция (Branch by abstraction)	Използване на блокове абстракция за постепенна замяна на функционалности.

Според бизнес функционалност

(Decompose by business
capability)

Този шаблон се фокусира върху разделяне на монолита на микросървиси според основните бизнес функции. Всеки микросървис обхваща отделна функционалност, като например "управление на поръчки", "фактуриране" или "плащания". Тези функции често могат да бъдат определени като части от бизнес процес или домейн.

Според бизнес функционалност



Според бизнес функционалност

(Decompose by business capability)

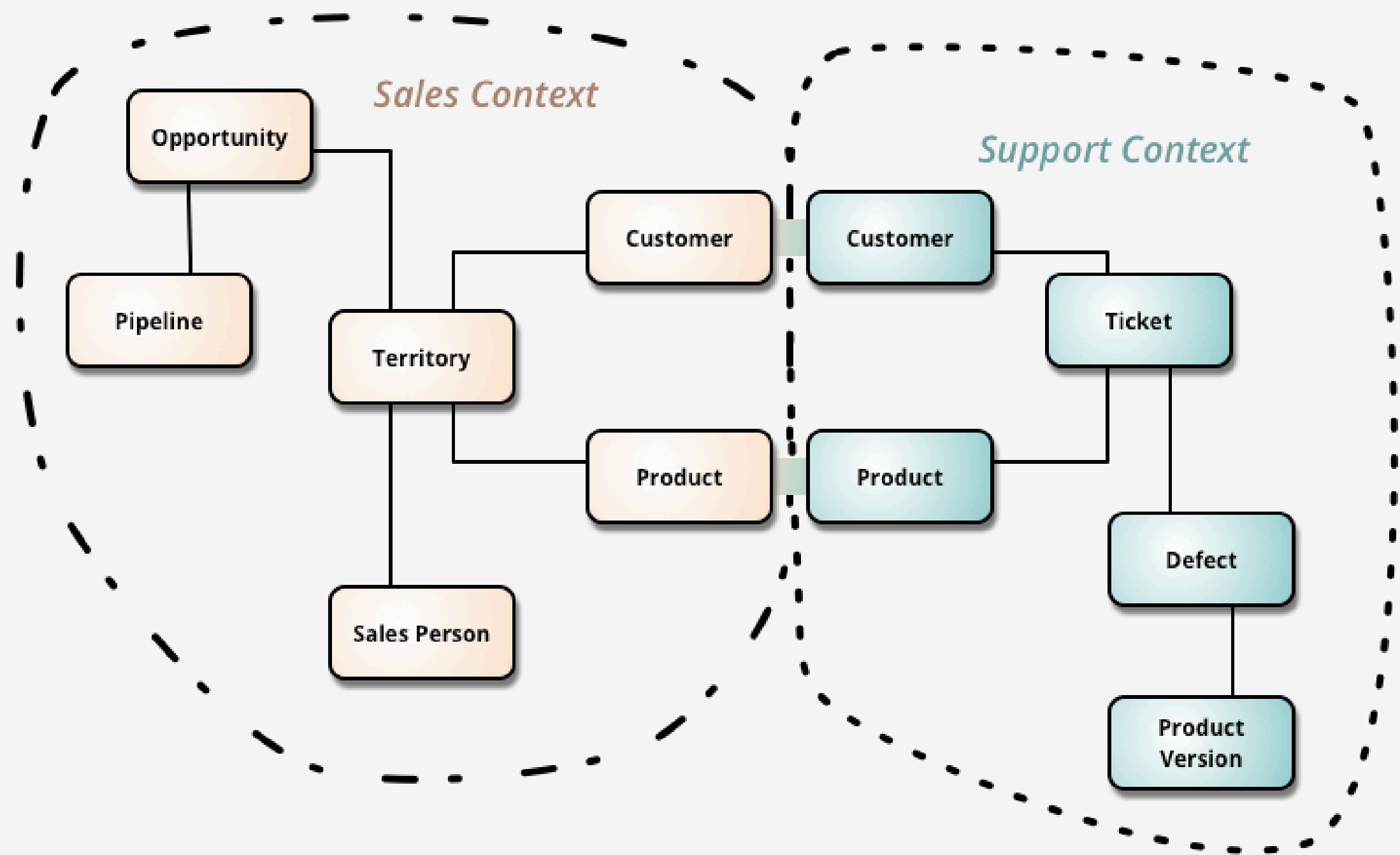
- **Кога е подходящ?** - Когато бизнес процесите са добре дефинирани и ясно разграничени. При наличие на екипи, които вече поддържат конкретни области (бизнес функции).
- **Предимства** - улеснява разделянето на работата между екипите. Микросървисите могат да се добавят или мащабират индивидуално за определен бизнес процес.
- **Предизвикателства** - изисква дълбоко разбиране на бизнес функциите в организацията. Може да доведе до сложни зависимости между микросървиси, свързани с общи процеси.

Снопег погдомейн

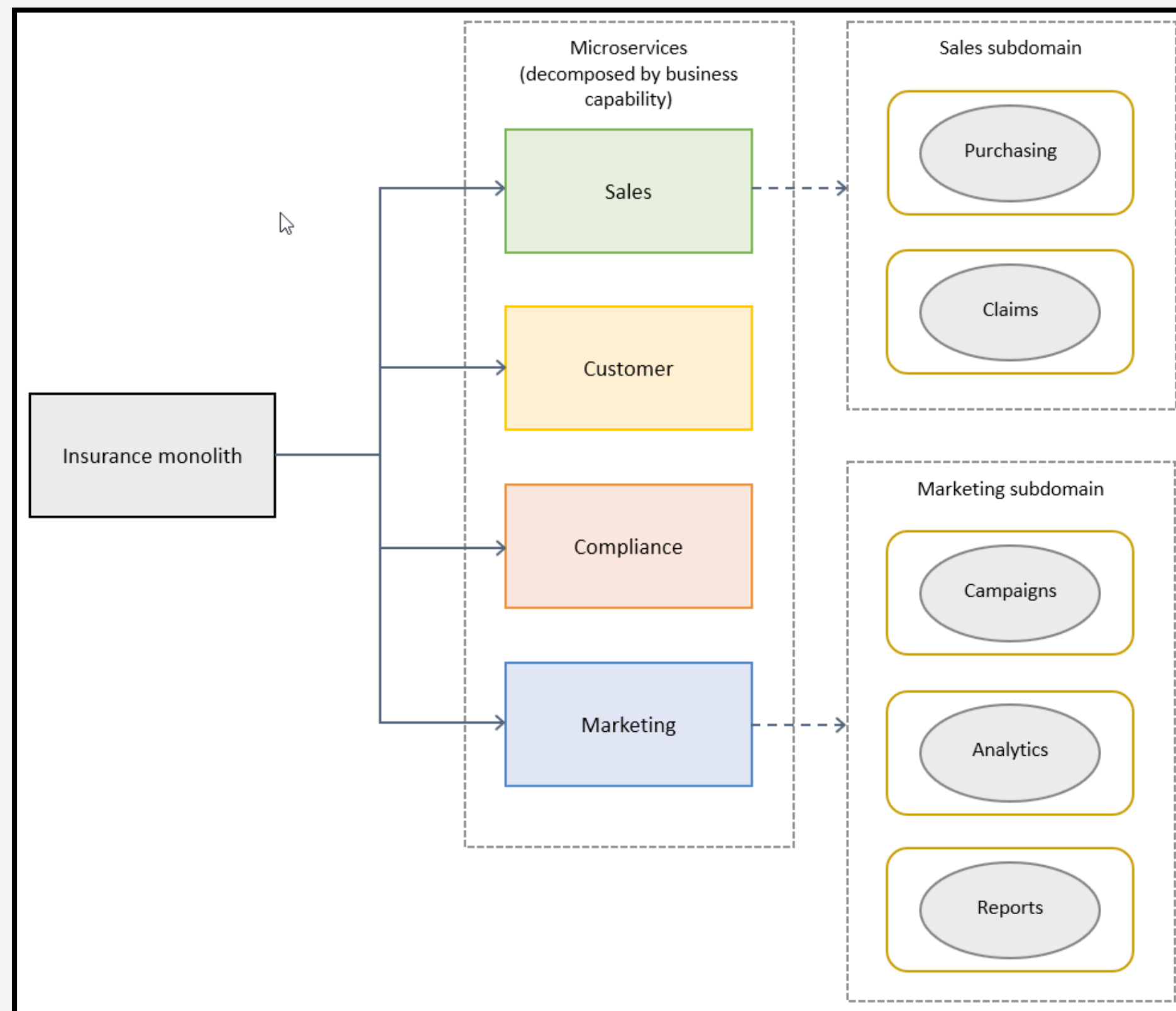
(Decompose by subdomain)

При този шаблон се използва методологията Domain-Driven Design (DDD), която разглежда системата като набор от домейни и поддомейни. Основната идея е да се изолират поддомейните с добре дефинирани граници (bounded contexts) и те да се реализират като микросървиси. Това е основен модел в дизайна, управляван от домейни. Той определя къде конкретни термини, модели и бизнес правила са валидни и как се използват, както и изрично посочва техните взаимовръзки.

Според поддомейн



Спорег погдомейн



Според поддомейн

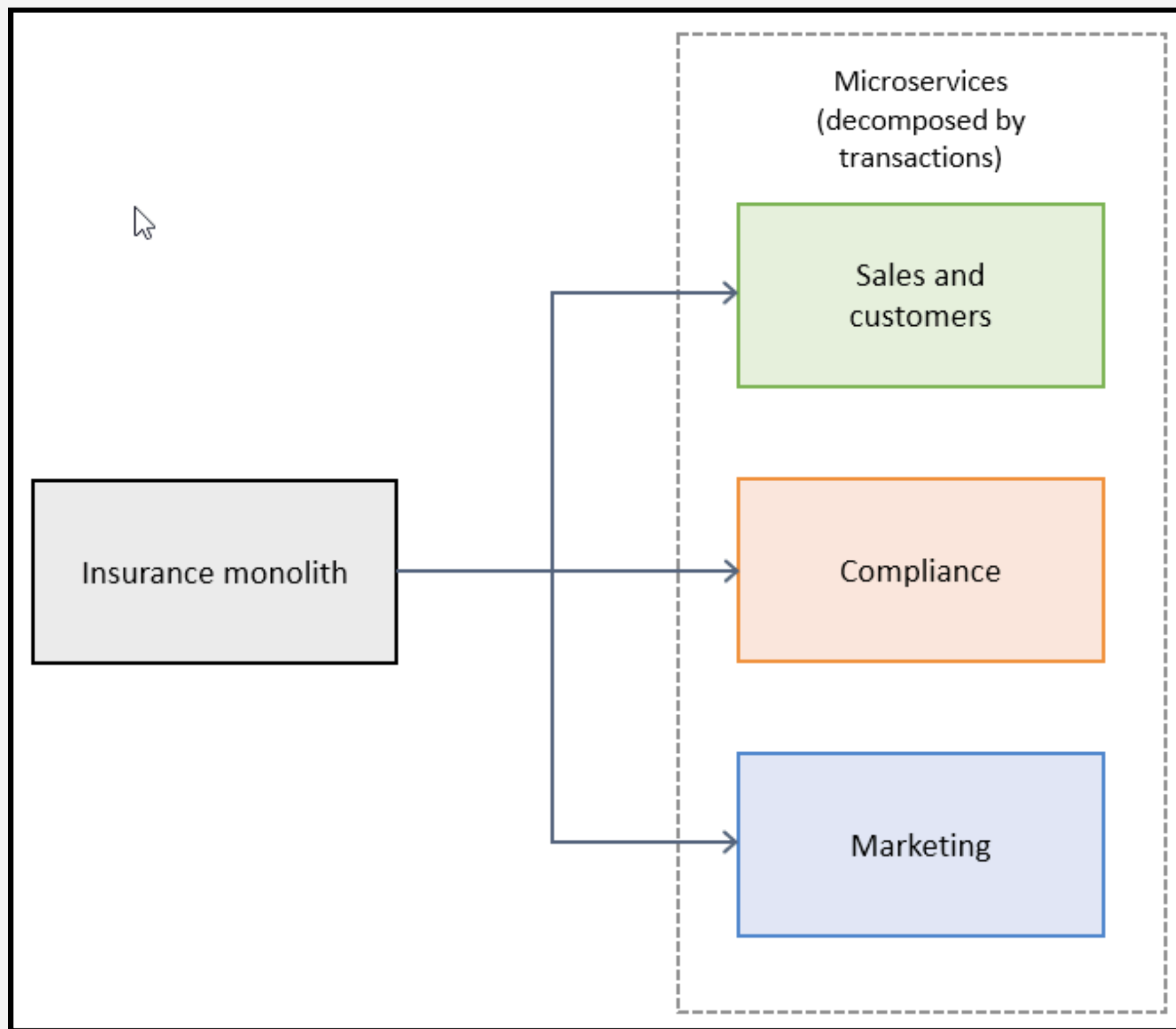
- **Кога е подходящ?** - когато имате комплексен домейн и искате да изолирате логиката на отделните части от него. За приложения, в които домейн знанието е силно доминиращо за архитектурните решения.
- **Предимства** - структурирано разделяне на логиката на базата на ясни контексти и отговорности. Намалява риска от дублиране на логика между микросървисите.
- **Предизвикателства** - изисква значително усилие за анализ на поддомейните и техните зависимости. Не е подходящо за системи с по-малка комплексност.

Според транзакции

(Decompose by transactions)

Този подход се основава на разделянето на монолита според зависимостите между данните (обикновено транзакции и обекти). Всеки микросървис управлява определен набор от транзакции (например "добавяне на продукти в количка" или "обработка на плащания").

Според транзакции



Според транзакции

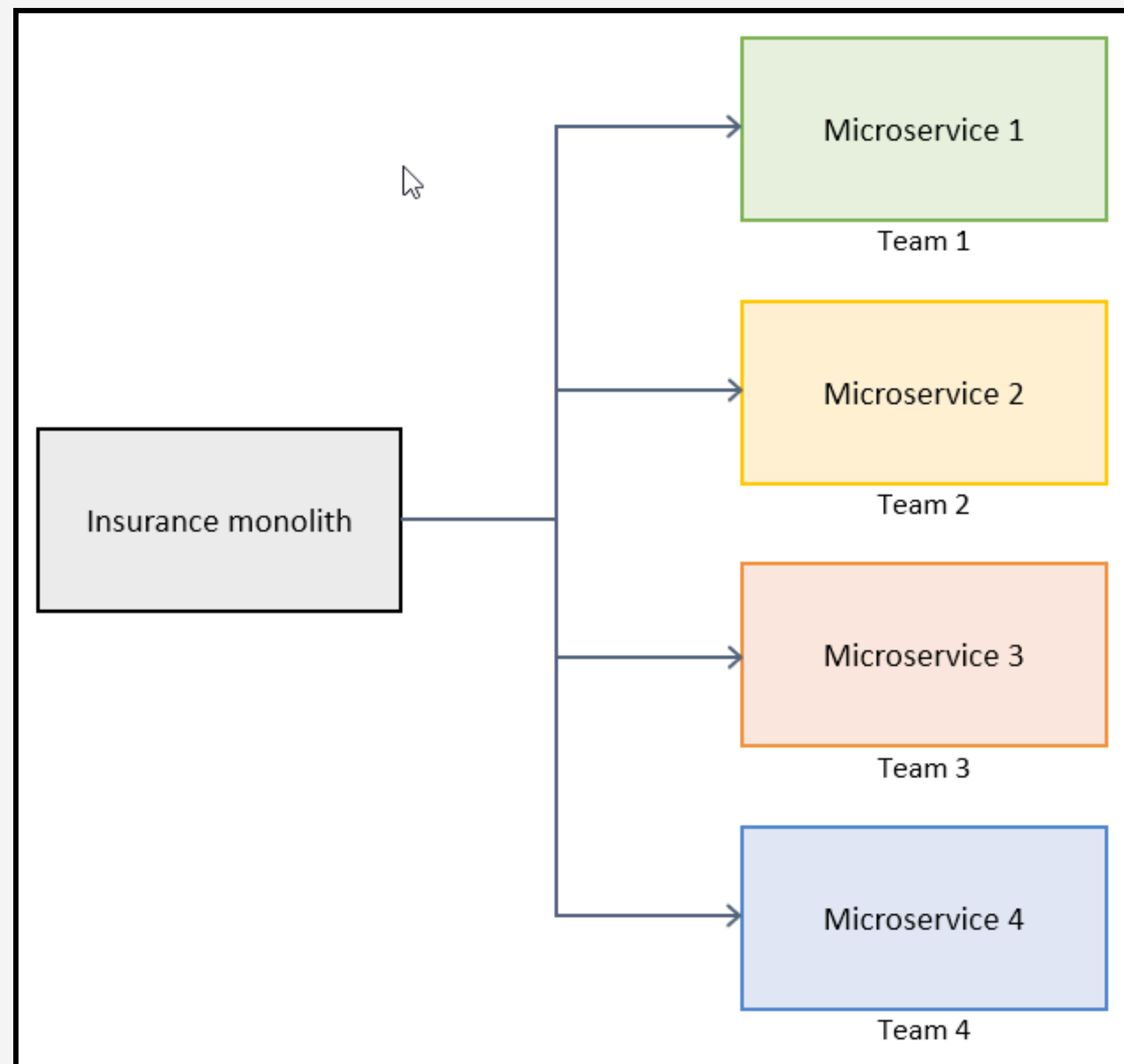
- **Кога е подходящ?** - Когато определени транзакции или операции често генерират натоварване и трябва да бъдат мащабирани независимо. За системи, които имат добре дефинирани граници между различни видове транзакции.
- **Предимства** - намалява свързаността между микросървисите, защото всяка услуга отговаря за своите операции.
- **Предизвикателства** - изисква внимателно идентифициране на взаимозависимостите между транзакциите. Може да доведе до сложност при координация на транзакции между множество услуги.

Един микросървис на екип

(Service per team pattern)

Този шаблон се основава на идеята, че всеки екип отговаря за конкретна част от системата и съответно трябва да поддържа свой собствен микросървис. Целта е организацията на разработката да се синхронизира с архитектурата на приложението.

Един микросървис на екип



Един микросървис на екип

- **Кога е подходящ?** - в големи организации с множество екипи, които работят по различни компоненти на системата. За приложения, където отговорностите могат да бъдат ясно разграничени.
- **Предимства** - екипите могат да работят автономно върху своите услуги. Улеснява разделението на работата и минимизира конфликти между билдовете.
- **Предизвикателства** - риск от размиване на границите между логиката на микросървисите, ако екипите не координират своите усилия. Може да доведе до прекомерно увеличаване на броя микросървиси в системата.

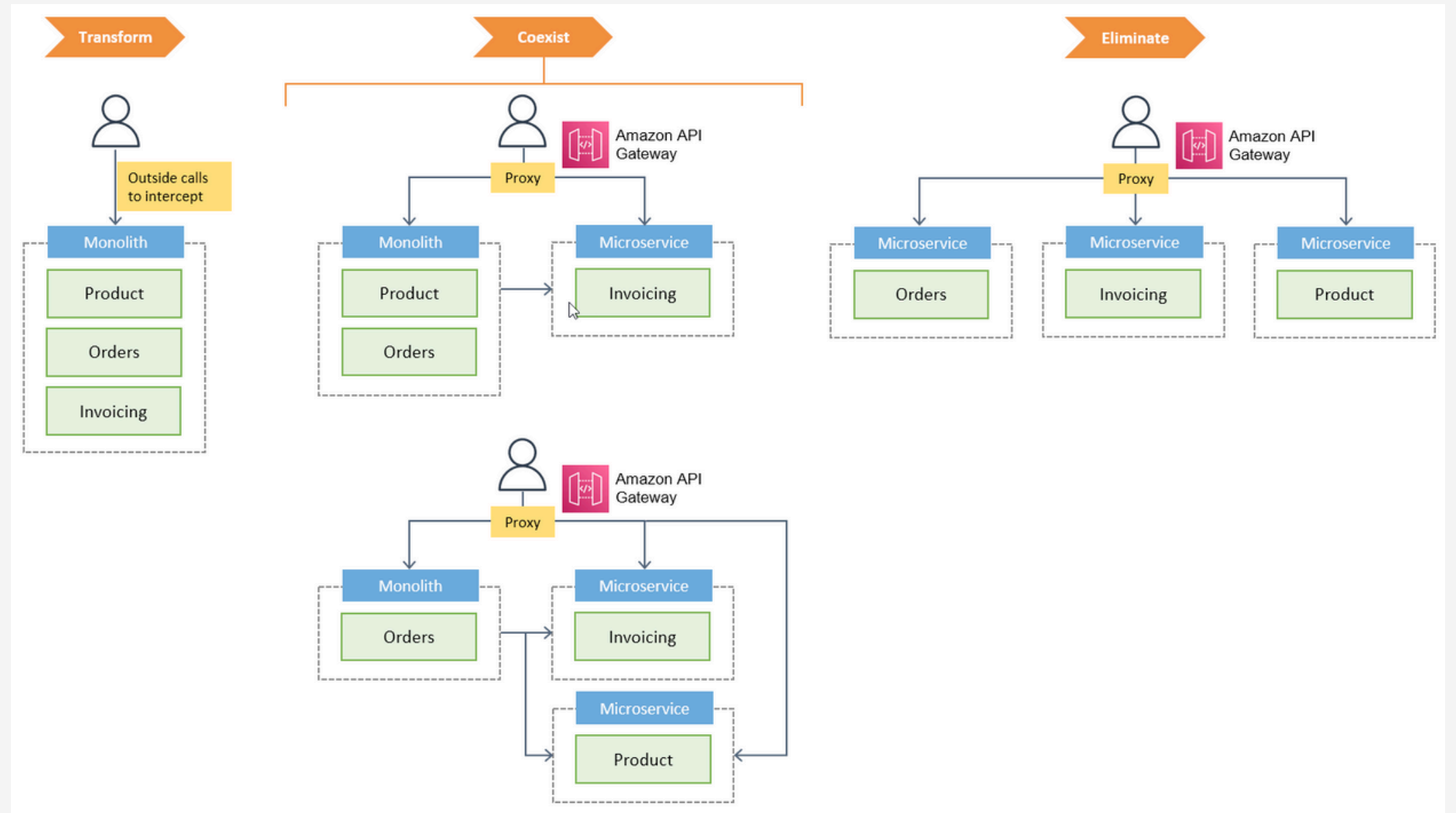
Шаблон

„Стрейнглър“

(Strangler Fig Pattern)

Този шаблон следва постепенен подход, при който се заместват части от монолита с нови микросървиси. Докато новите услуги се създават, монолитът продължава да съществува паралелно, докато не бъде напълно премахнат.

Шаблон „Стрейнглър“



Шаблон „Стрейнгълър“

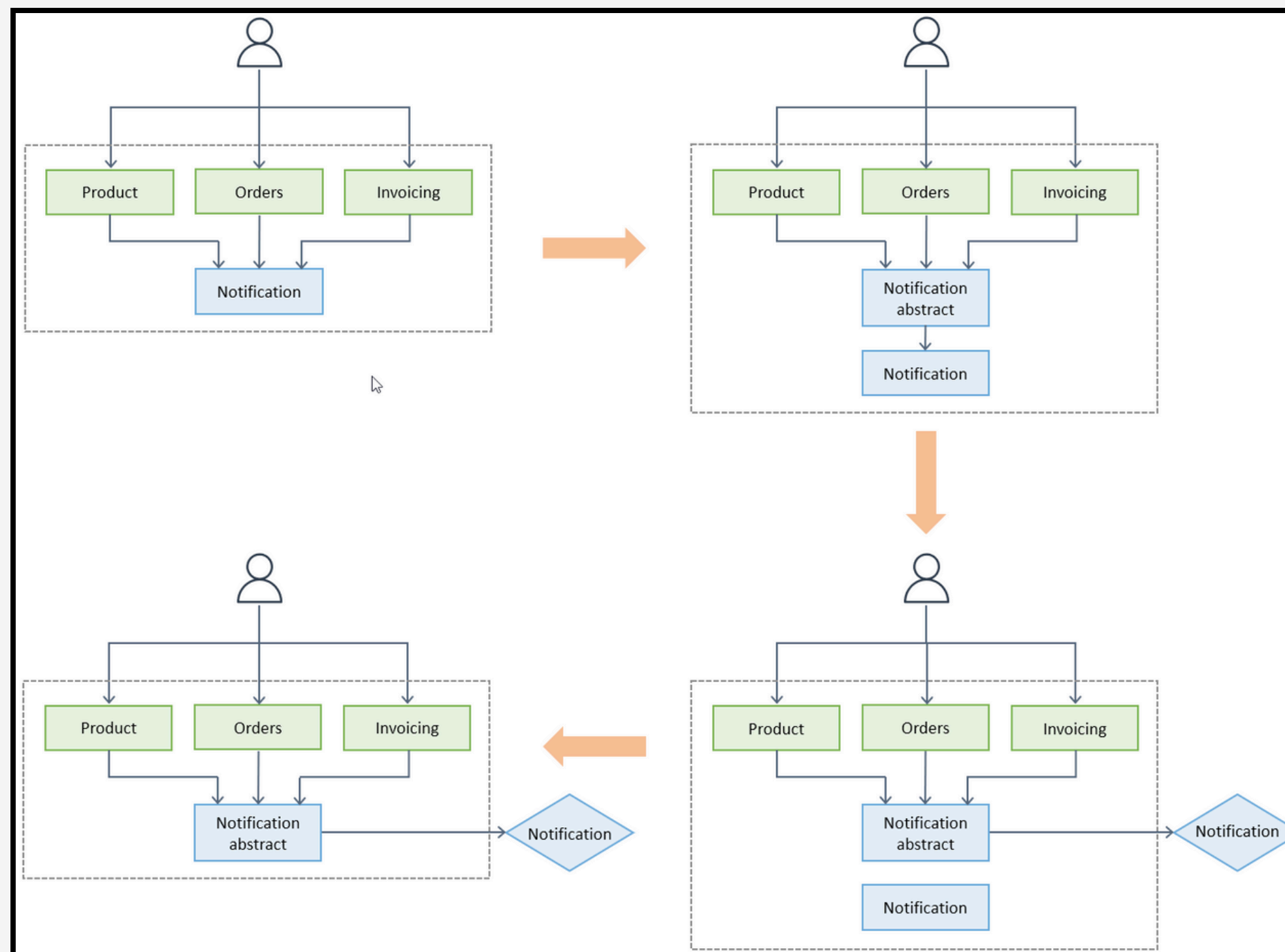
- **Кога е подходящ?** - за наследени системи (legacy systems), където е рисковано монолитът да бъде изцяло премахнат наведнъж. За големи и критични приложения, които не могат да бъдат спрени.
- **Предимства** - минимизира риска, тъй като трансформацията става постепенно. Остава време архитектурните решения да бъдат адаптирани при нужда.
- **Предизвикателства** - изисква управление на двоен технологичен стек (монолит + микросървиси), докато миграцията приключи. Временните интеграции между монолита и микросървисите могат да доведат до сложност.

Разклоняване чрез абстракция

(Branch by abstraction)

Този шаблон насърчава внедряването на абстракции, които позволяват поетапна промяна на вътрешни компоненти или функционалности, без да възпрепятстват текущото функциониране на системата.

Разклоняване чрез абстракция (Branch by abstraction)



Разклоняване чрез абстракция

- **Кога е подходящ?** - при нужда от внедряване на промени в критични системи, които трябва да останат оперативни. За проекти, където интеграцията с нови технологии или архитектури трябва да се случи плавно.
- **Предимства** - значително намалява риска при трансформацията. Осигурява обратимост при миграцията, ако се открият проблеми.
- **Предизвикателства** - увеличава архитектурната сложност поради поддържането на временни абстракции. Може да изисква допълнителна координация между екипите.

Разделяне на базата данни

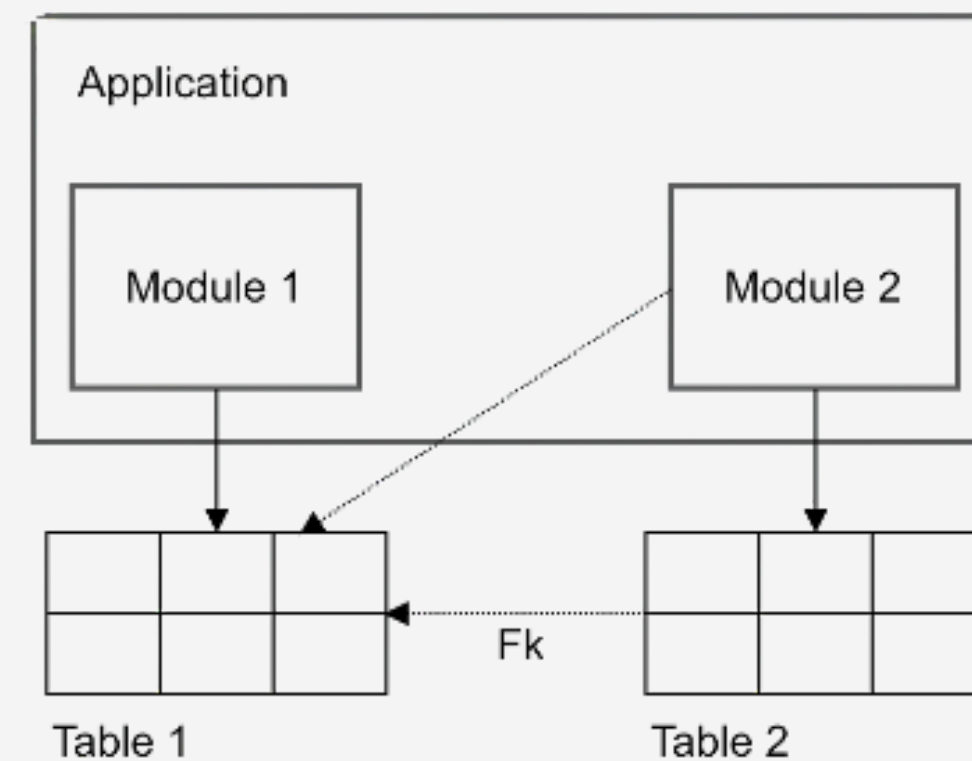
Разделянето на базата от данни е една от критичните стъпки при процеса на декомпозиция на монолитните приложения. Тази стъпка изисква внимателно разглеждане на начина, по който данните са организирани, достъпвани и свързани в текущата система. Липсата на добре планирано разделяне може да доведе до трудности при извличане на информация или до нарушаване на основната функционалност.

Разделяне на базата данни

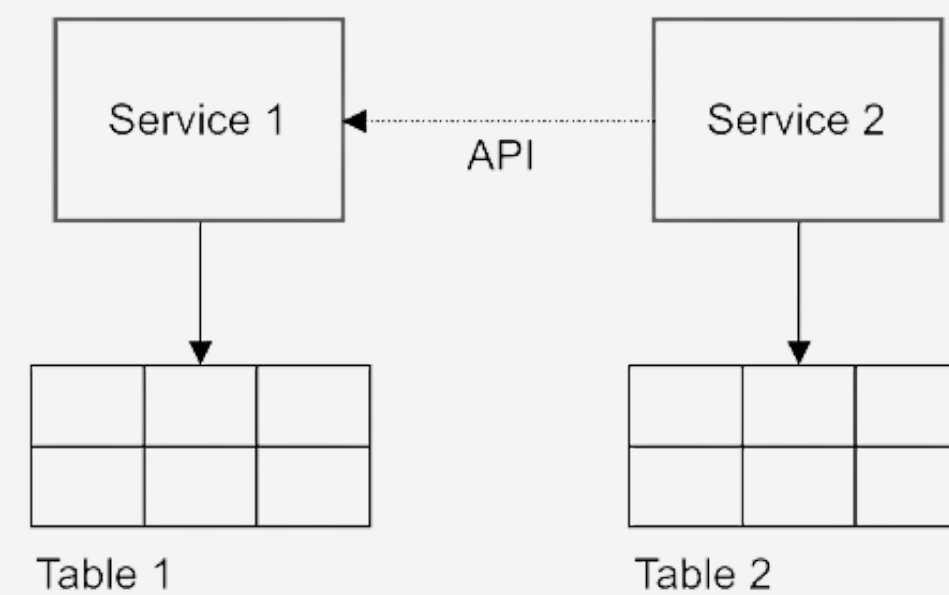
Един от големите проблеми, който трябва да решим, е загубата на релациите между таблиците. Защото една таблица може да е част от една услуга, а друга, която консумира информация от нея, посредством външен ключ (foreign key), да е в друга услуга и съответно друга база от данни

Разделяне на базата данни

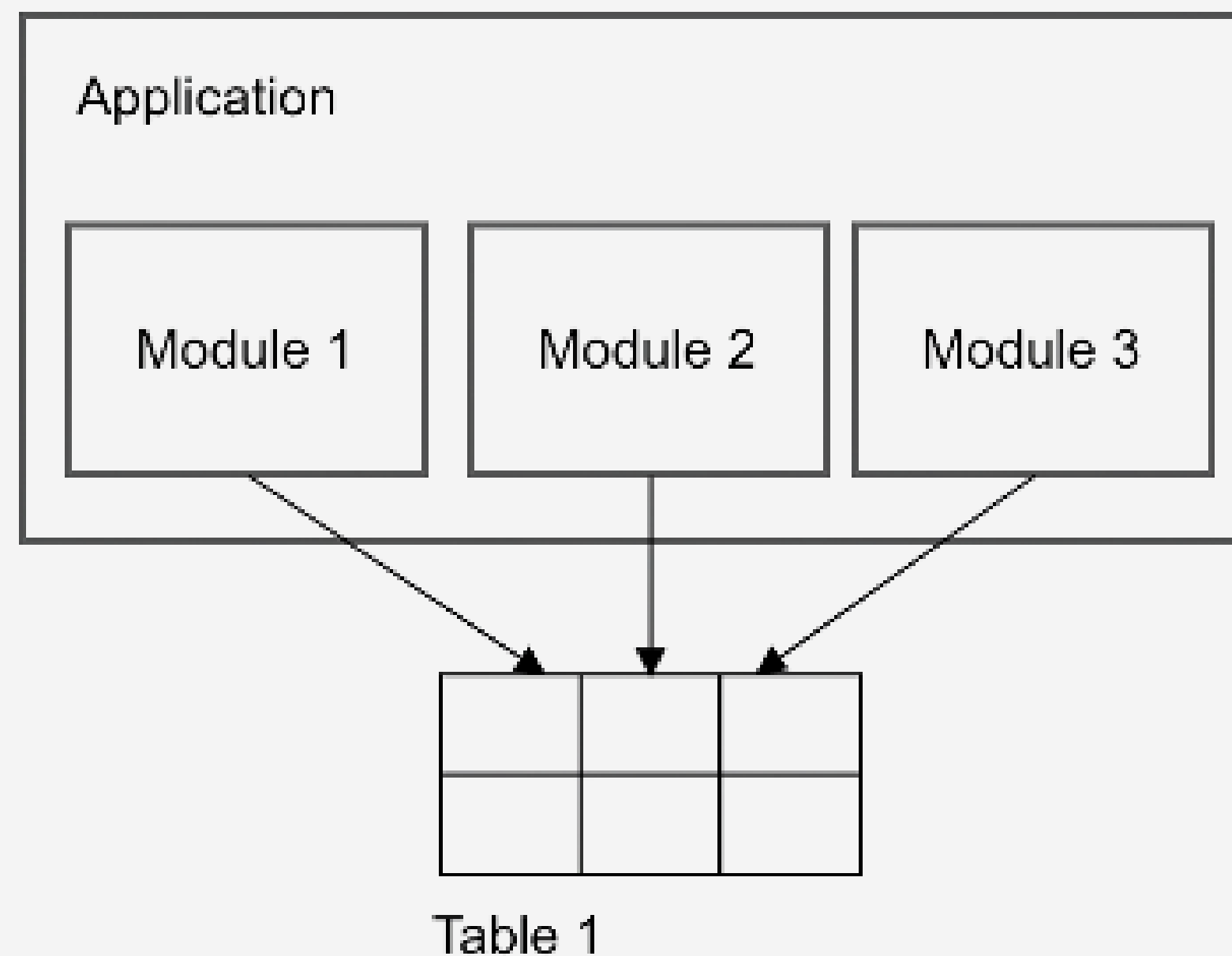
Преди



След



Споделени данни между различни модули

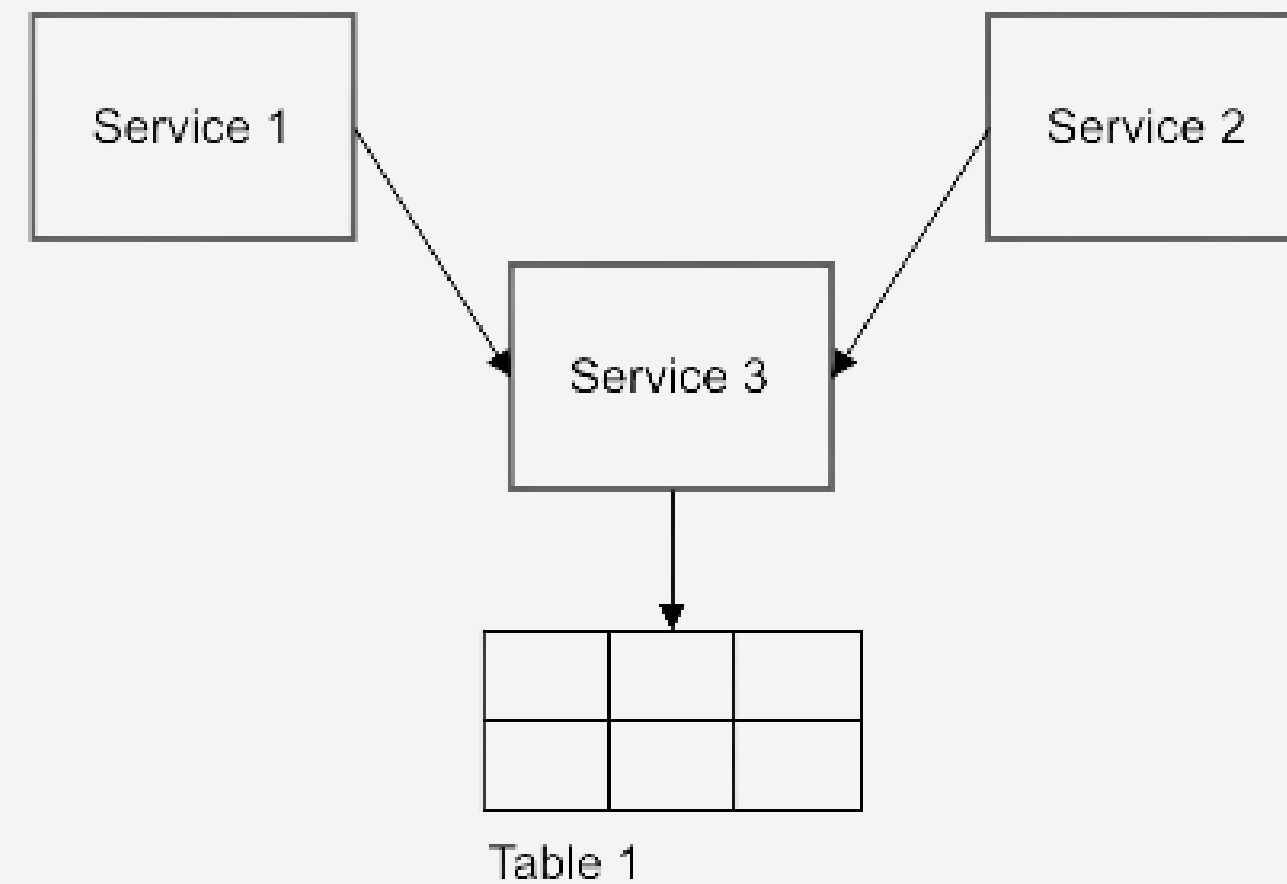
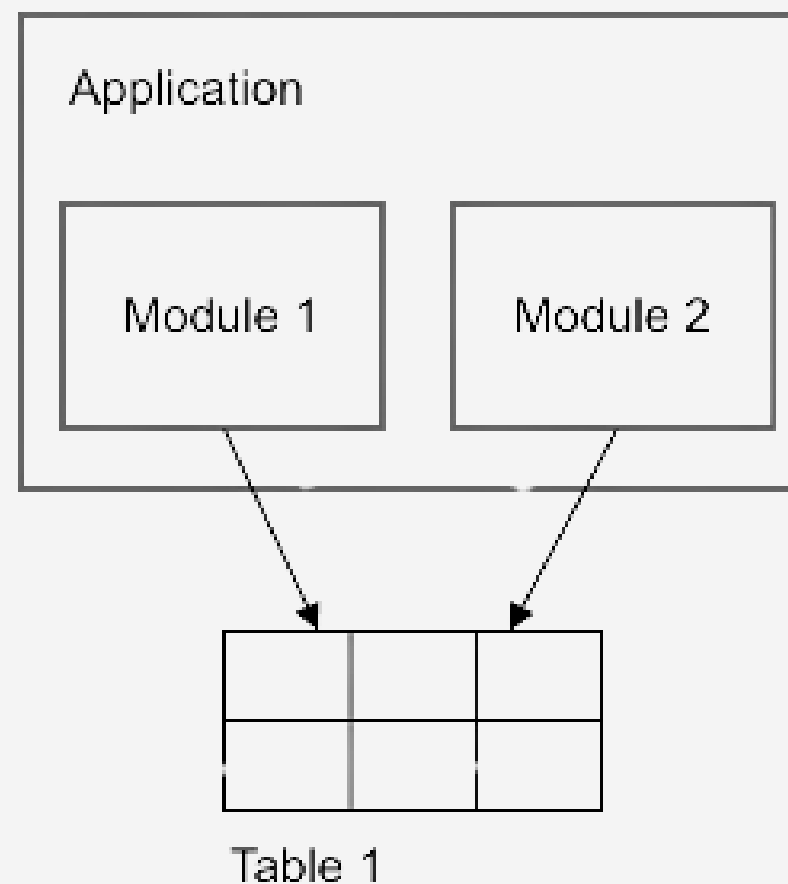


Споделени данни между различни модули

При споделени данни между различни модули могат да се приложат следните подходи:

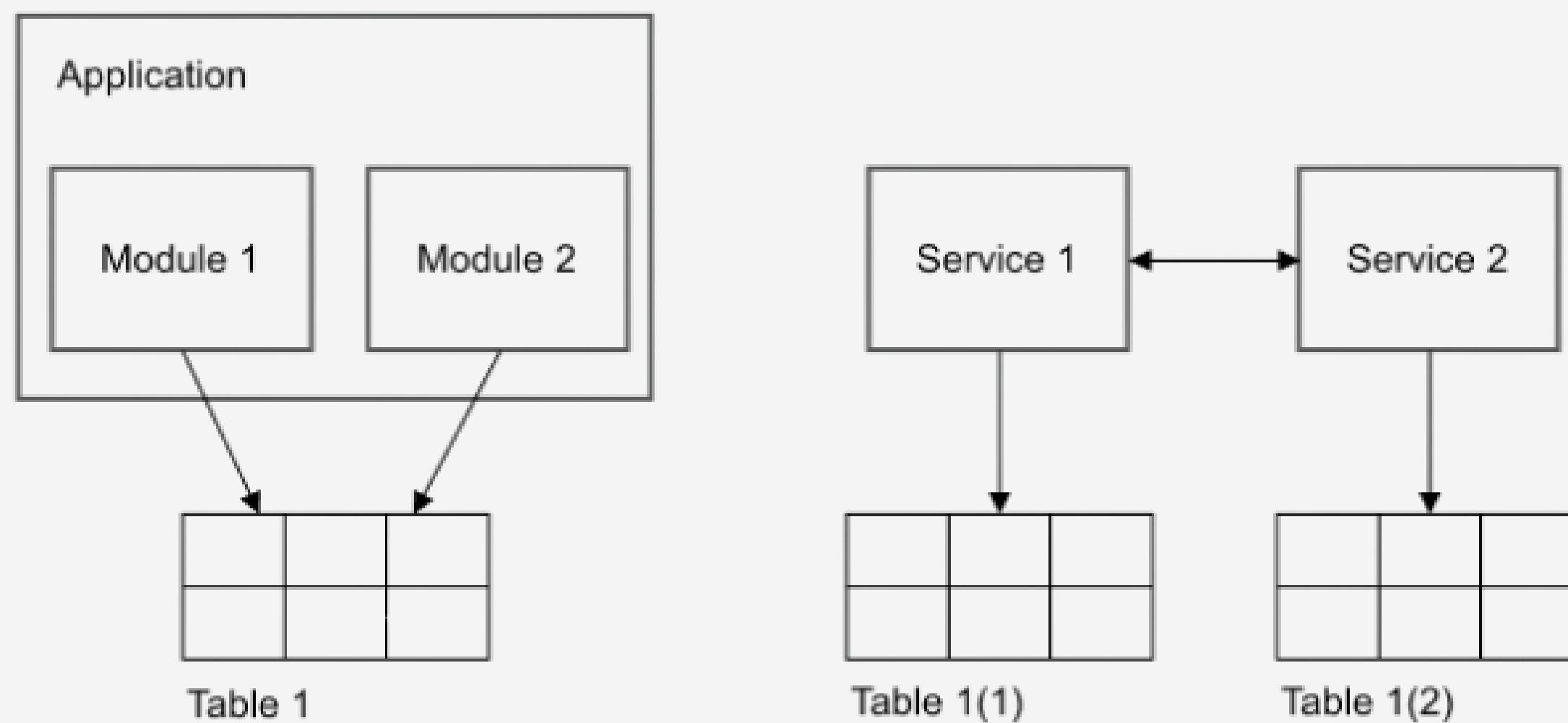
1. Дублиране на данни - информацията се копира в базите от данни на отделните услуги. Това решение изисква механизъм за синхронизиране на промените, за да се избегнат несъответствия.
2. Създаване на централизирана услуга - обособява се отделна услуга, която управлява и предоставя достъп до споделените ресурси (например справочни таблици с държави, пощенски кодове и др.).
3. Предоставяне на данни извън база от данни - споделената информация се съхранява извън базите, например като част от конфигурацията на услугите или в споделен сървър за кеширане (cache server).

Споделяне на данни между модули



При споделена таблица, която е единствена точка на истината (Single Point of Truth) за два модула, се създава нова услуга, отговорна за поддържане на данните, която предоставя API за достъп и актуализация.

Споделяне на данни между модули



Таблицата може да се раздели на две независими таблици в различни бази, а липсващата информация да се извлича чрез API.